Basic Quantitative Method: TA Session Week 1

Danny Po-Hsien Kang (康柏賢)

July 3, 2026

TA Session Website

- All materials in the TA Session can be found in here
  - Including slides, practices, etc.

Today's agenda

1. Introduction to TA Session
2. Brief Introduction to R
3. Download R & RStudio
4. Basics of R
   - RStudio Interface
   - Assign Values
   - Data Types
   - Download Packages
   - Help Function
5. R Guides & Tips / Reading CSVs
   - Open a script
   - `getwd()`, `setwd()`
   - Project
   - `read.csv()`
   - Folder Organization
   - Coding styles

What will we do in the TA session?

1. Learn how to use R
   - Basic syntax, Data manipulation, Data visualization, etc.
2. In-class practice
   - Some practices will be prepared to check your understanding in each TA session.
3. R Programming Challenges

How to contact me?

1. Sending email
   - Please mail to r13323002@ntu.edu.tw

What is R/RStudio?

- **R** is a free and open-source programming language for **statistical computing** and **data analysis**.
- **RStudio** is an integrated development environment (IDE) for R.

Icon for R

Icon for RStudio

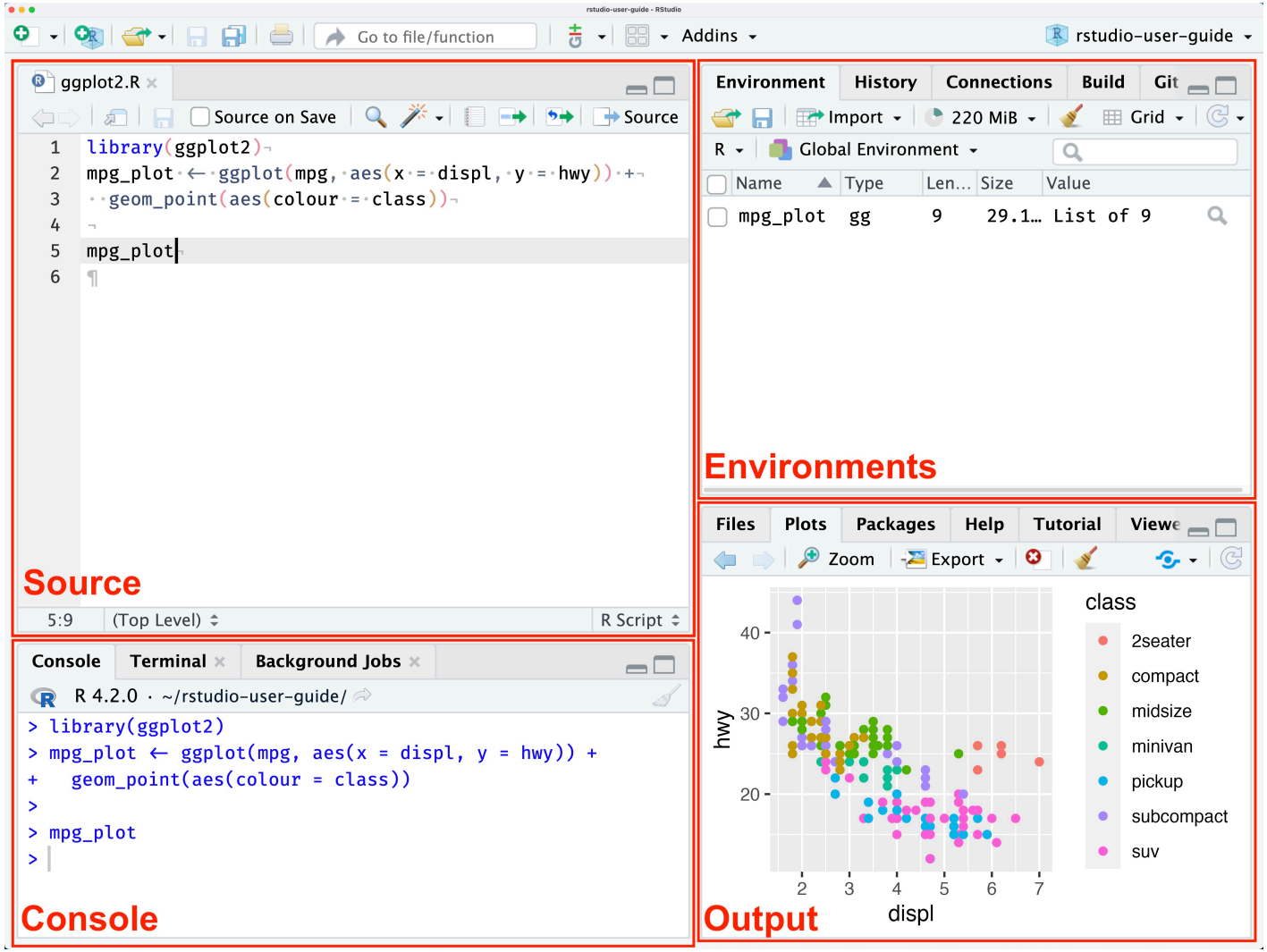
Why choose R?

Aspect	R	Stata	Python
Costs	Free, open-source	Paid	Free, open-source
Primary uses	Data analysis/ Statistics	Data analysis/ Statistics	Various purposes
Learning	Moderate	Easier	Moderate
Typical users	Econ/Stats/Soc. sci.	Econ/Stats/Soc. sci.	Popular in industries

Download R & RStudio

- Download R here.
- Download RStudio here.
- Install R first, and then install RStudio.

Basics of R: RStudio Interface



Basics of R: RStudio Interface

- **Source** (Script): write your code here.
- **Environment**: check your data and objects here.
- **Console**: execute your code here.
- **Files/Plots/Packages/Help**: check your files, plots, packages, and help here.

Basics of R: Assign Values

- <- is the **assignment operator** in R. We use it to assign value to a variable.

```
# example
a <- 3
```

- A value assigned to a variable can be a number, a character, a dataframe, or the result of an expression.

```
# example
b <- a*a
# b = 9
x <- "hello" # x = "hello"
```

Basics of R: Data Types

- There are several basic data types in R:
  - **Numeric**: numbers with decimal points.
  - **Character**: text
  - **Logical**: `TRUE` or `FALSE`.
- To check the data type of a variable, use the `class()` function.

```
# example
 x <- 5
 class(x) # Output: "numeric"
 y <- "Hello World!"
 class(y) # Output: "character"
 z <- TRUE
 class(z) # Output: "logical"
```

Basics of R: Data Types

- To store data in R, we have the following ways:
  - **Vector**: one-dimensional sequence of values of the same data type.
  - **List**: one-dimensional sequence of values (could) with different data types.
  - **Matrix**: two-dimensional structure of values of the same data type.
  - **Data frame**: two-dimensional structure of values (could) with different data types across columns.

Basics of R: Data Types (Vector)

- Examples of `vector`: use `x:y` or `c()` to create vector

```
id <- 1:4 # 1 2 3 4
class(id)
# Output: "integer"
score <- c(30, 86, 50, 95)
class(score) # Output: "numeric"
name <- c("Sam", "John", "Andy", "Judy")
class(name) # Output: "character"
```

Basics of R: Data Types (List)

- Examples of `list`:

```
my_list <- list("Hello World!", 1, TRUE, score)
```

- Output

```
> print(my_list)
[[1]]
[1] "Hello World!"

[[2]]
[1] 1

[[3]]
[1] TRUE

[[4]]
[1] 30 86 50 95
```

Basics of R: Data Types (Matrix)

- Examples of `matrix(data, nrow=..., ncol=...)`:[1]

```
id <- 1:4
score <- c(30, 86, 50, 95)
id_score <- matrix(c(id, score), nrow = 4)
```

- Output

```
> print(id_score)
     [,1] [,2]
[1,]    1   30
[2,]    2   86
[3,]    3   50
[4,]    4   95
```

[1] Also try what happens when without `nrow` and when `nrow = 2`!

Basics of R: Data Types (Data Frame)

- Examples of `data.frame()`:

```
df <- data.frame(stu_id = id,
                 stu_score = score)
```

- Output

```
> print(df)
  stu_id stu_score
1      1        30
2      2        86
3      3        50
4      4        95
```

Basics of R: Download Packages

- To download and install a package, use the `install.packages()` function.

```
install.packages("tidyverse")
```

- To load a package into your R session, use the library() function.

```
library(tidyverse)
```

- Add `""` when use `install.packages()` install, but no need when use `library()`.

Basics of R: Help Function

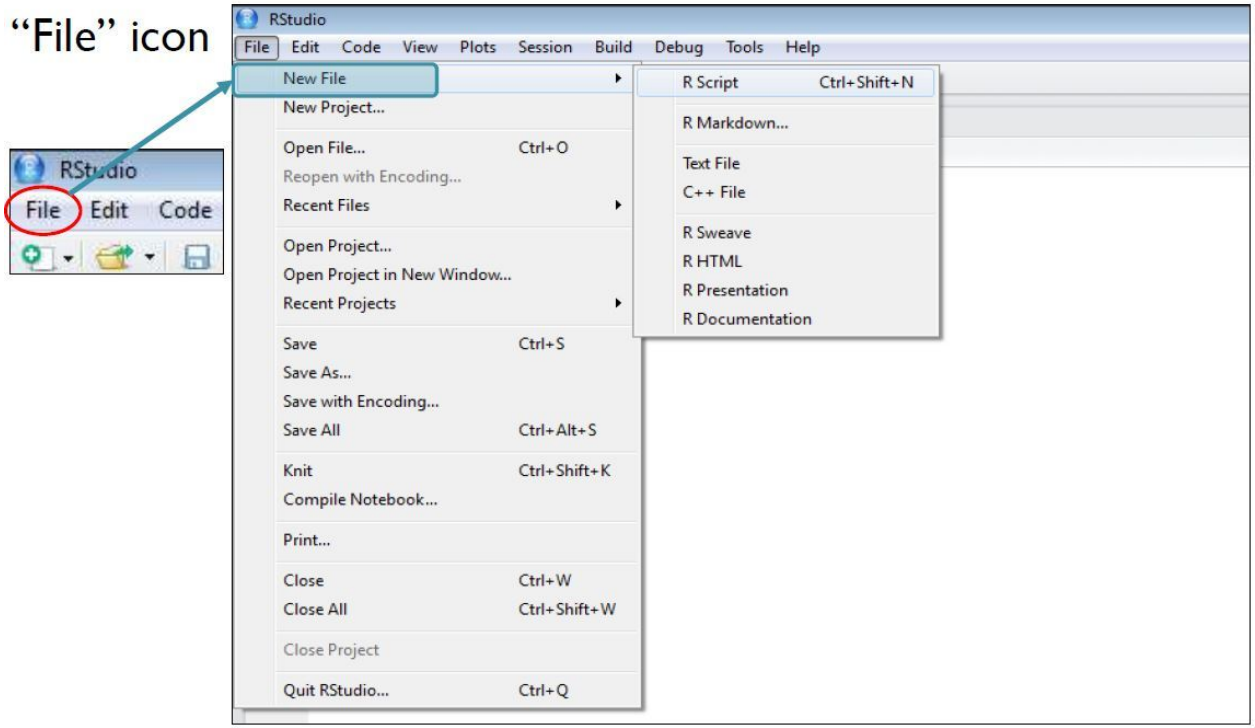
- To get help on a specific function, use the `help()` function or the `?` operator to check the documentation for the function.

```
help(mean)
?mean
```

- This will open a help page with information about the function, its arguments, and examples of usage.
- Other types of help function: `help.start()` and `help.search()`.

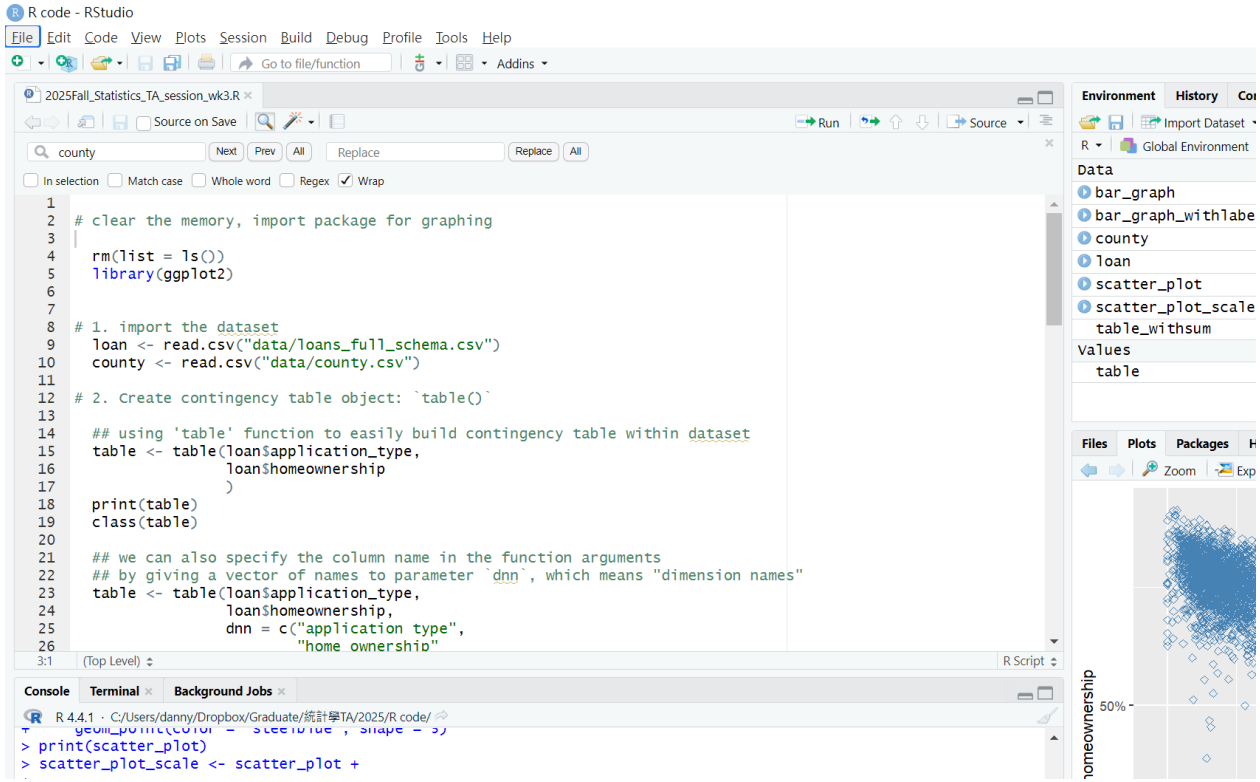
Script in RStudio

- A **script** is a text file that you can write your code in.



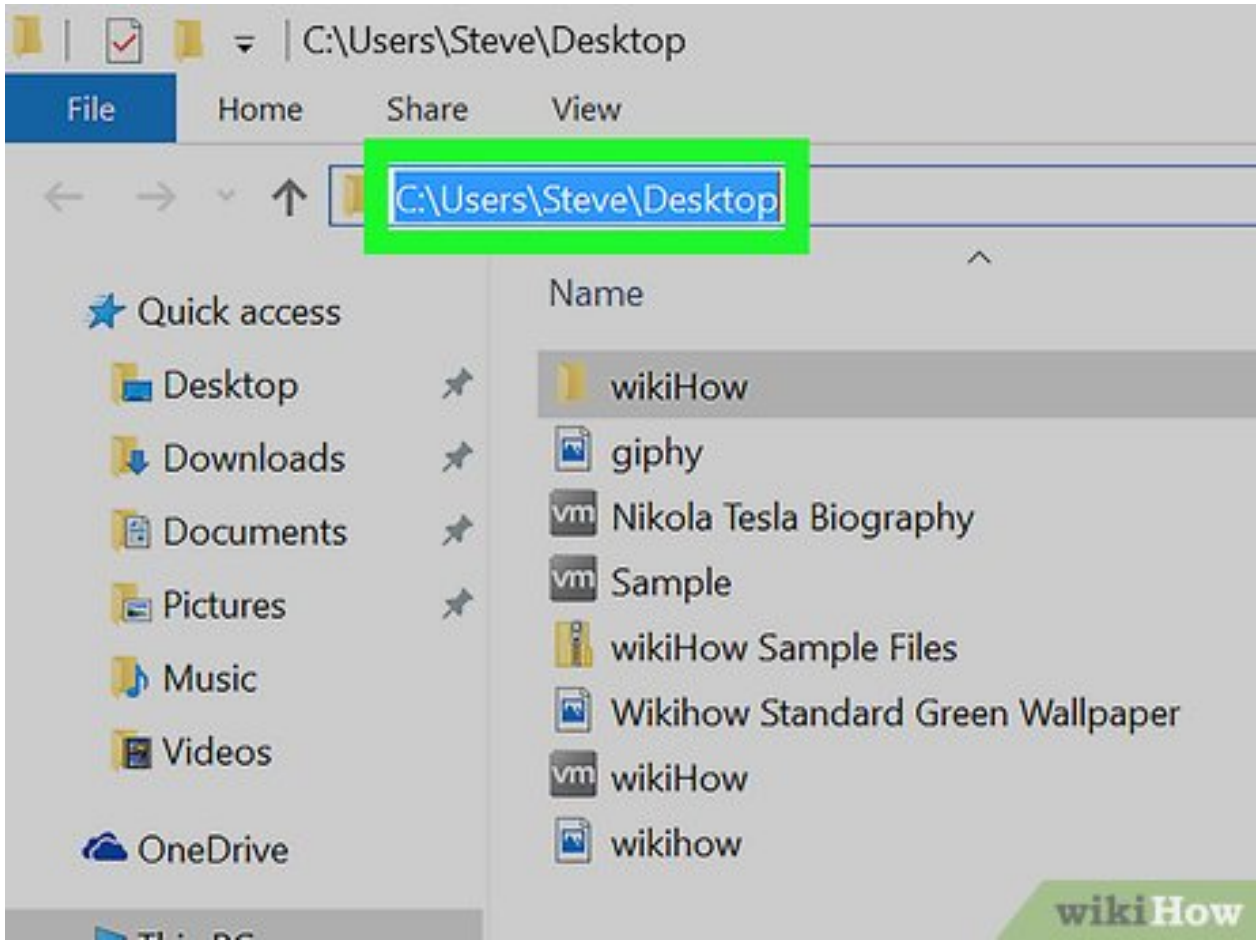
Why should we open a script?

- Main Reason: We open an R script so we can save it!
- Save your code in the script such that it is executable and reproducible.



Path of a file

- A **path** for a file is the place where you save it.



Get/Set Current Directory

- `getwd()`: Get current working directory

```
getwd() #Example Output: "C:/your/working/directory"
```

- `setwd()`: Change your working directory[2]

```
setwd("C:\your\working\directory") #Error!
setwd("C:\\your\\working\\directory") #Works on Windows!
setwd("C:/your/working/directory") #Works everywhere!
```

[2] You can use `dir.exists()` to check if the directory exists.

RStudio project

- **RStudio project**: A tool to organize all files related to a task.
  - Sets the root folder as the wd for all R sessions within the project.
  - No more `setwd()`!
  - Keep history, options ... etc.
  - More advantages. e.g. portable path: `here()` function.
- Open a project by (File → New Project) at the top left corner of RStudio.
- When should I open a RStudio project?
  - If the task lasts for multiple periods, or with multiple files.
- Recommend to open a RStudio project for the TA Session Practices!

Read CSV File: read.csv()

- **Syntax:**

```
df <- read.csv("file.csv") #Work if file in working directory
df <- read.csv("folder_name/file.csv") #If file is in folder
df <- read.csv("C:/your/working/directory/file.csv")
```

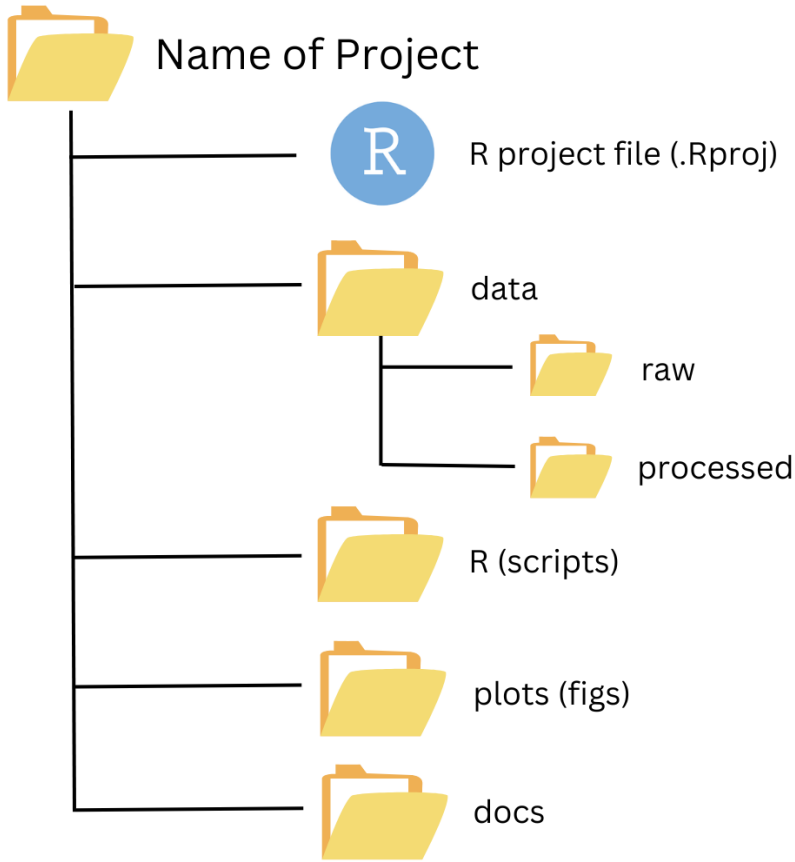
- **Example:**

```
babies <- read.csv("C:/your/working/directory/babies.csv")

# Do this if you have your R project created!
earthquake <- read.csv("data/earthquake.csv")
```

Folder Organization

- After opening a project, it is important to have well-organized folders.
  - Different types of things (scripts, data, output, etc.) should be saved in different folders!



Coding styles: Naming

- Only lowercase letters, numbers, and _ within a name.
- The name should be readable![3]

```
# Good
lesson_one
lesson_1

# Bad
LessonOne
lessonone
one
a <- 1
```

[3] See this website for more on coding styles.

Coding styles: Spacing

- Always put a space after a comma, never before.
- Do not put spaces inside or outside parentheses.

```
# Good
x[, 1]
mean(x, na.rm = TRUE)

# Bad
x[,1]
x[ ,1]
x[ , 1]
mean (x, na.rm = TRUE)
mean( x, na.rm = TRUE )
```

Coding styles: Function Indents

- Indent the argument name with a single indent (two spaces).
- Don't put + or , in a new line!

```
# Good
long_function_name <- function(
  a = "a long argument",
  b = "another argument",
  c = "another long argument"
) {
}
# Bad
long_function_name <- function(a = "a long argument",
  b = "another argument",
  c = "another long argument"
) {
}

long_function_name <- function(a = "a long argument", b = "anot
```

Practice Session

- We will have some R practices after each lecture.
  - Practices can be found in the TA Session's Website: here.
- Raise your hand/come to TA if you finish or have any questions on the practices.
  - Feel free to leave after finishing them.